CLOUD & DEVOPS

Feature Flags Visual Guide

Decouple deploy from release — roll out with a kill switch in hand.

ANIMATED WALKTHROUGH

Step through pages to play the animation

STEP 1/6

Decouple Deploy from Release

Ship code dark, then turn it on for the right people.

Deploy ≠ release. Code can ride the train for weeks before anyone sees it.

Trunk-based development loves flags: merge small, keep main deployable, release on your schedule.

STEP 2/6

Targeting Rules

Who sees the flag? Rules decide, per request.

Percentage

10% of users (sticky by user ID)

Attributes

plan=enterprise AND region=eu

Allowlist

internal team + beta customers first

Gradual

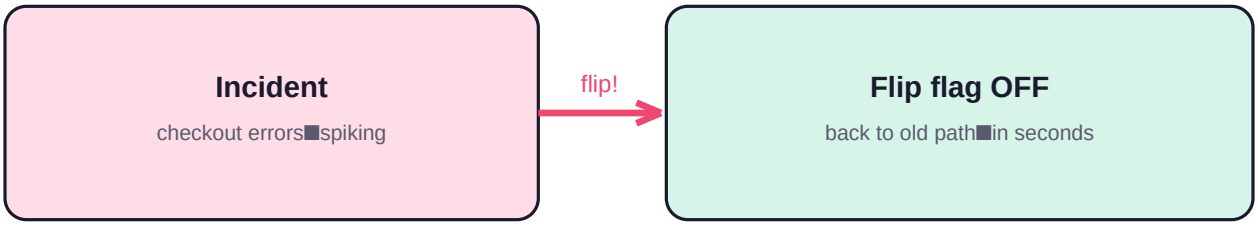
5% → 25% → 50% → 100% on a schedule

Sticky bucketing: hash(user_id) → same user always sees the same variant. No flicker.

STEP 3/6

The Kill Switch

When the new checkout breaks at 2am, you don't redeploy — you flip.



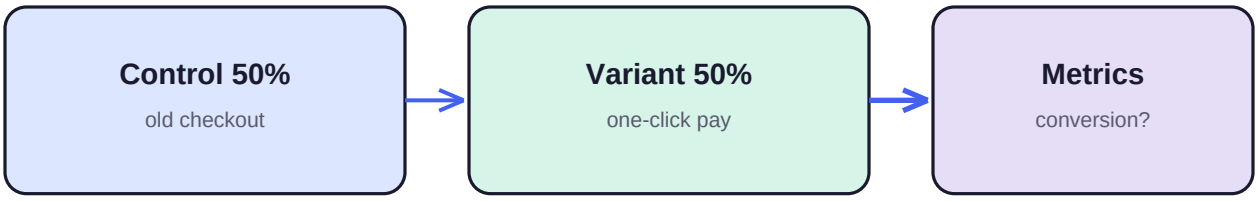
Recovery time: seconds, not the 20 minutes a rollback deploy takes.

- Every risky feature ships behind a flag — the flag IS the rollback plan.
- Flag changes must be audited: who flipped what, when (it's a production change).
- Test the OFF path too — the kill switch is useless if the old code rotted.

STEP 4/6

Gradual Rollouts & Experimentation

Flags double as the control plane for A/B tests.



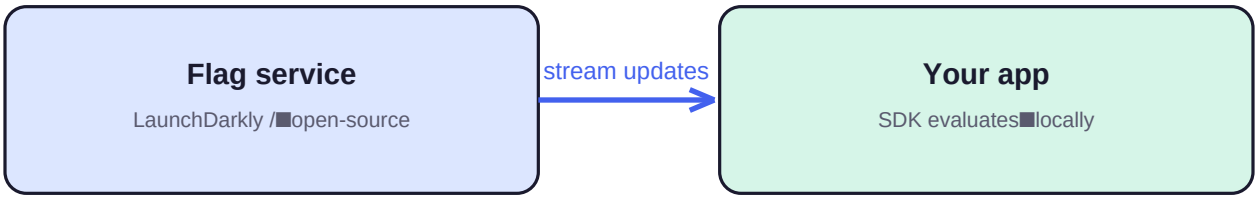
Same machinery as targeting — but the goal is measurement, not safety.

One variable per experiment. Flag the UI copy AND the backend behavior together or results lie.

STEP 5/6

Flag Architecture

SDKs evaluate locally; a control plane manages state.



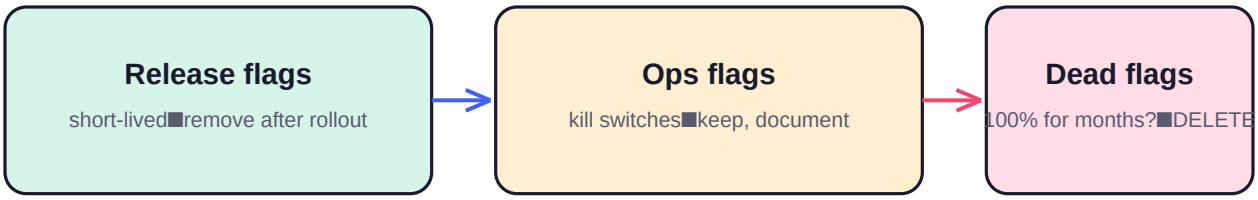
Evaluation is LOCAL (cached rules, microseconds) — the service is only the control plane.

- SDKs cache flag state and stream changes — a flag-service outage must not break your app.
- Default to the SAFE value when the SDK can't reach the service (usually OFF).
- Bootstrap flags at startup so first requests don't block on a network call.

STEP 6/6

Tech Debt: Flags Rot

Every flag is a branch in production — clean them up.



Rule: a flag that hit 100% gets a removal ticket the same sprint. Stale flags are untested code paths.

STEP ✓/✓

Key Takeaways

Feature Flags — Remember

- Deploy ≠ release: ship dark, release to the right audience on your schedule.
- Target by percentage, attributes, allowlists — sticky bucketing, no flicker.
- The kill switch is the fastest rollback: seconds, audited, tested both ways.
- Same machinery powers A/B tests — one variable, metrics decide.
- Flags rot: remove release flags after rollout; dead flags are untested branches.

Keep exploring

Pair this guide with the matching video walkthroughs and hands-on labs at interstitiumlabs.dev